NIBRAS PLATFORM

REST API Reference

Complete documentation for all 97 endpoints — authentication, project tracking, submissions, team formation, program planning, notifications, and administrative operations.

BASE URL
`http://localhost:3001`

VERSION
`v2.0.0`

REVISED
May 2026

ENDPOINTS
97 total

Contents

01	Authentication & Sessions	10 endpoints
02	System & Health	5 endpoints
03	GitHub Integration	5 endpoints
04	CLI Projects & Submissions	9 endpoints
05	Tracking — Courses & Members	14 endpoints
06	Tracking — Projects & Milestones	14 endpoints
07	Tracking — Submissions & Reviews	9 endpoints
08	Tracking — Team Formation	7 endpoints
09	Tracking — Dashboards & Analytics	7 endpoints
10	Program Planning	22 endpoints
11	Notifications	6 endpoints
12	Admin	13 endpoints

Quick-Reference: All Endpoints

Method colours: GET Read POST Create / Action PATCH Update DELETE Delete

All endpoints are prefixed with the base URL. Endpoints marked 🔒 Auth require either a `Authorization: Bearer <token>` header (CLI) or a `nibras-session` cookie (web). Endpoints marked 🔓 Public are unauthenticated.

Quick-Reference: All Endpoints

Method	Path	Auth	Tag
GET	/healthz	🔓 Public	system
GET	/readyz	🔓 Public	system
GET	/metrics	🔓 Public	system
GET	/v1/health	🔓 Public	system
GET	/v1/ping	🔓 Public	system
POST	/v1/device/start	🔓 Public	auth
POST	/v1/device/poll	🔓 Public	auth
POST	/v1/device/authorize	🔒 Auth	auth
GET	/v1/github/oauth/start	🔓 Public	auth
GET	/v1/github/oauth/callback	🔓 Public	auth
POST	/v1/auth/refresh	🔓 Public	auth
POST	/v1/logout	🔒 Auth	auth
GET	/v1/me	🔒 Auth	auth
GET	/v1/web/session	🔒 Auth	auth
POST	/v1/web/logout	🔒 Auth	auth
DELETE	/v1/me/account	🔒 Auth	auth
GET	/v1/github/config	🔓 Public	github
GET	/v1/github/install-url	🔒 Auth	github
POST	/v1/github/setup/complete	🔒 Auth	github
POST	/v1/github/repositories/validate	🔒 Auth	github
POST	/v1/github/webhooks	🔓 Public	github
GET	/v1/projects/:key/manifest	🔒 Auth	projects
GET	/v1/projects/:key/task	🔒 Auth	projects
GET	/v1/projects/:key/starter-bundle	🔒 Auth	projects
POST	/v1/projects/:key/setup	🔒 Auth	projects
POST	/v1/submissions/prepare	🔒 Auth	projects
POST	/v1/submissions/:id/local-test-result	🔒 Auth	projects
GET	/v1/submissions/:id	🔒 Auth	projects
GET	/v1/submissions/:id/stream	🔒 Auth	projects
GET	/v1/me/submissions	🔒 Auth	projects
GET	/v1/tracking/courses	🔒 Auth	tracking

Method	Path	Auth	Tag
POST	/v1/tracking/courses	🔒 Auth	tracking
GET	/v1/tracking/courses/:id/members	🔒 Auth	tracking
POST	/v1/tracking/courses/:id/members	🔒 Auth	tracking
DELETE	/v1/tracking/courses/:id/members/:uid	🔒 Auth	tracking
PATCH	/v1/tracking/courses/:id/members/:uid/level	🔒 Auth	tracking
POST	/v1/tracking/courses/:id/invites	🔒 Auth	tracking
POST	/v1/tracking/courses/:id/invites/bulk	🔒 Auth	tracking
GET	/v1/tracking/invites/:code	🔓 Public	tracking
POST	/v1/tracking/invites/:code/join	🔒 Auth	tracking
GET	/v1/tracking/courses/:id/templates	🔒 Auth	tracking
POST	/v1/tracking/courses/:id/templates	🔒 Auth	tracking
GET	/v1/tracking/courses/:id/projects	🔒 Auth	tracking
GET	/v1/tracking/courses/:id/export.csv	🔒 Auth	tracking
POST	/v1/tracking/projects	🔒 Auth	tracking
GET	/v1/tracking/projects/:id	🔒 Auth	tracking
PATCH	/v1/tracking/projects/:id	🔒 Auth	tracking
POST	/v1/tracking/projects/:id/publish	🔒 Auth	tracking
POST	/v1/tracking/projects/:id/unpublish	🔒 Auth	tracking
GET	/v1/tracking/templates/:id	🔒 Auth	tracking
PATCH	/v1/tracking/templates/:id	🔒 Auth	tracking
GET	/v1/tracking/projects/:id/milestones	🔒 Auth	tracking
POST	/v1/tracking/projects/:id/milestones	🔒 Auth	tracking
GET	/v1/tracking/milestones/:id	🔒 Auth	tracking
PATCH	/v1/tracking/milestones/:id	🔒 Auth	tracking
DELETE	/v1/tracking/milestones/:id	🔒 Auth	tracking
GET	/v1/tracking/milestones/:id/submissions	🔒 Auth	tracking
POST	/v1/tracking/milestones/:id/submissions	🔒 Auth	tracking
GET	/v1/tracking/submissions/:id	🔒 Auth	tracking
PATCH	/v1/tracking/submissions/:id	🔒 Auth	tracking
DELETE	/v1/tracking/submissions/:id	🔒 Auth	tracking
GET	/v1/tracking/submissions/:id/commits	🔒 Auth	tracking
GET	/v1/tracking/submissions/:id/review	🔒 Auth	tracking
POST	/v1/tracking/submissions/:id/review	🔒 Auth	tracking
GET	/v1/tracking/review-queue	🔒 Auth	tracking

Method	Path	Auth	Tag
POST	/v1/tracking/projects/:id/applications	🔒 Auth	tracking
GET	/v1/tracking/projects/:id/applications/me	🔒 Auth	tracking
GET	/v1/tracking/projects/:id/applications	🔒 Auth	tracking
POST	/v1/tracking/projects/:id/team-formation/generate	🔒 Auth	tracking
POST	/v1/tracking/projects/:id/team-formation/lock	🔒 Auth	tracking
GET	/v1/tracking/projects/:id/teams	🔒 Auth	tracking
PATCH	/v1/tracking/projects/:id/teams/:tid	🔒 Auth	tracking
GET	/v1/tracking/dashboard/home	🔒 Auth	tracking
GET	/v1/tracking/dashboard/student	🔒 Auth	tracking
GET	/v1/tracking/dashboard/instructor	🔒 Auth	tracking
GET	/v1/tracking/dashboard/course/:id	🔒 Auth	tracking
GET	/v1/tracking/analytics/student	🔒 Auth	tracking
GET	/v1/tracking/analytics/instructor	🔒 Auth	tracking
GET	/v1/tracking/activity	🔒 Auth	tracking
GET	/v1/programs	🔒 Auth	programs
POST	/v1/programs	🔒 Auth	programs
GET	/v1/programs/:id/versions/:vid	🔒 Auth	programs
POST	/v1/programs/:id/versions	🔒 Auth	programs
POST	/v1/programs/:id/catalog-courses	🔒 Auth	programs
POST	/v1/programs/:id/requirement-groups	🔒 Auth	programs
PATCH	/v1/programs/:id/requirement-groups/:gid	🔒 Auth	programs
POST	/v1/programs/:id/tracks	🔒 Auth	programs
PATCH	/v1/programs/:id/tracks/:tid	🔒 Auth	programs
POST	/v1/programs/:id/enroll	🔒 Auth	programs
GET	/v1/programs/student/me	🔒 Auth	programs
PATCH	/v1/programs/student/me/plan	🔒 Auth	programs
GET	/v1/programs/student/me/recommend-track	🔒 Auth	programs
POST	/v1/programs/student/me/select-track	🔒 Auth	programs
GET	/v1/programs/student/me/sheet	🔒 Auth	programs
POST	/v1/programs/student/me/generate-sheet	🔒 Auth	programs
POST	/v1/programs/student/me/petitions	🔒 Auth	programs
GET	/v1/programs/student/me/petitions	🔒 Auth	programs
GET	/v1/programs/:id/petitions	🔒 Auth	programs
PATCH	/v1/programs/:id/petitions/:pid	🔒 Auth	programs

Method	Path	Auth	Tag
POST	/v1/programs/:id/approvals/:spid/advisor	🔒 Auth	programs
POST	/v1/programs/:id/approvals/:spid/department	🔒 Auth	programs
GET	/v1/notifications	🔒 Auth	notifications
GET	/v1/notifications/count	🔒 Auth	notifications
POST	/v1/notifications/read-all	🔒 Auth	notifications
PATCH	/v1/notifications/:id/read	🔒 Auth	notifications
GET	/v1/notifications/preferences	🔒 Auth	notifications
PATCH	/v1/notifications/preferences/:type	🔒 Auth	notifications
GET	/v1/admin/submissions	🔒 Auth	admin
PATCH	/v1/admin/submissions/:id/status	🔒 Auth	admin
GET	/v1/admin/submissions/:id/logs	🔒 Auth	admin
POST	/v1/admin/submissions/:id/retry	🔒 Auth	admin
POST	/v1/admin/submissions/bulk-retry	🔒 Auth	admin
GET	/v1/admin/audit-logs	🔒 Auth	admin
GET	/v1/admin/projects	🔒 Auth	admin
DELETE	/v1/admin/courses/:id	🔒 Auth	admin
POST	/v1/admin/projects/:id/archive	🔒 Auth	admin
GET	/v1/admin/users	🔒 Auth	admin
PATCH	/v1/admin/users/:id/role	🔒 Auth	admin
GET	/v1/admin/students	🔒 Auth	admin
PATCH	/v1/admin/students/:id/year	🔒 Auth	admin

FEATURE AREA

Authentication & Sessions

All CLI clients authenticate using a **GitHub Device Flow**. The web dashboard uses server-side session cookies. Tokens are JWTs and must be refreshed with the refresh token when expired.

Two auth schemes are accepted on authenticated endpoints:

- **CLI clients** — `Authorization: Bearer <accessToken>` header
- **Web clients** — `nibras-session` HttpOnly cookie (set on login)

Device Flow (CLI)

POST `/v1/device/start` 🔓 Public

Begin GitHub device flow. Returns a user-facing code and verification URL. The CLI displays this code and opens the browser.

200 Device codes issued

```
{ deviceCode: string,        // internal code — do not show
  userCode: string,           // show to user (e.g. "ABCD-1234")
  verificationUri: string,    // https://github.com/login/device
  verificationUriComplete: string,
  intervalSeconds: number,    // poll interval (default 5 s)
  expiresInSeconds: number }
```

POST `/v1/device/poll` 🔓 Public

Poll for authorization result. Call every `intervalSeconds` until status is `authorized` or `expired`.

PARAMETERS

Name	Type	Description
`deviceCode`	`string`	The device code from /device/start

200 Pending / success

```
// Still waiting:
{ status: "pending" | "expired" | "denied" }

// Authorized:
{ status: "authorized",
  accessToken: string,
  refreshToken: string,
  user: { id, username, email, githubLogin,
          githubLinked, githubAppInstalled,
          systemRole: "user"|"admin", yearLevel } }
```

POST `/v1/device/authorize` 🔒 Auth

Authorize a pending device code using the current web session. Called automatically by the web UI after the user approves the device.

PARAMETERS

Name	Type	Description
`userCode`	`string`	The 8-character code shown to the user

200 `{ ok: true }`

Web OAuth

GET `/v1/github/oauth/start` 🔓 Public

Redirect the browser to GitHub OAuth. Begins web dashboard login. Accepts `?return_to=<url>` query parameter.

302 Redirect to GitHub

GET `/v1/github/oauth/callback` 🔓 Public

GitHub OAuth callback — handled server-side. Sets the `nibras-session` cookie and redirects back to the app.

302 Redirect to return_to URL

Token Management

POST `/v1/auth/refresh` 🔓 Public

Exchange a refresh token for a new access + refresh token pair. CLI calls this automatically on 401 responses.

PARAMETERS

Name	Type	Description
`refreshToken`	`string`	Current refresh token

200 New token pair

```
{ accessToken: string, refreshToken: string }
```

POST `/v1/logout` 🔒 Auth

Revoke a CLI Bearer session token. The refresh token is invalidated.

200 `{ ok: true }`

User Identity

GET `/v1/me` 🔒 Auth

Get the currently authenticated CLI user, API base URL, and course memberships.

200 User object

```
{ user: { id, username, email, githubLogin, githubLinked,
          githubAppInstalled, systemRole, yearLevel },
  apiBaseUrl: string,
  memberships: [{ courseId, role, level }] }
```

GET `/v1/web/session` 🔒 Auth

Get the currently authenticated web user (session cookie). Same shape as GET /v1/me.

200 Same as GET /v1/me

POST `/v1/web/logout` 🔒 Auth

Revoke the current web session cookie.

200 `{ ok: true }`

DELETE `/v1/me/account` 🔒 Auth

Permanently delete the authenticated user's account (GDPR erasure). Anonymises all submissions, revokes all tokens, and removes personal data. This action is irreversible.

200 `{ ok: true }`

Warning: This endpoint deletes all user data permanently. Submissions are anonymised, not deleted, to preserve course records.

FEATURE AREA

System & Health

Health and metrics endpoints used by infrastructure monitoring. No authentication required.

GET `/healthz` 🔓 Public

Kubernetes liveness probe. Returns plain text `ok` when the process is alive.

200 `"ok"`

GET `/readyz` 🔓 Public

Kubernetes readiness probe. Verifies database connectivity before returning ok.

200 Ready

```
{ status: "ok" }
```

503 Database unavailable

```
{ status: "error", detail: string }
```

GET `/metrics` 🔓 Public

Prometheus-compatible metrics in Prometheus exposition format (plain text). Includes request counts, latencies, and DB pool stats.

200 Plain text metrics

GET `/v1/health` 🔓 Public

Simple API health check.

200 `{ status: "ok" }`

GET `/v1/ping` 🔓 Public

Extended ping. Returns richer information when an auth token or session is provided — GitHub link and App installation status.

200

```
{ ok: boolean,
  api: boolean,
```

```
  auth: boolean,            // true if valid token supplied
  githubLinked: boolean,
  githubAppInstalled: boolean }
```

FEATURE AREA

GitHub Integration

Endpoints for GitHub App configuration, installation, repository validation, and inbound webhook processing.

GET `/v1/github/config` 🔓 Public

Return GitHub App configuration status. Used by the CLI to show whether the platform has a GitHub App installed.

200

```
{ configured: boolean,
  appName: string | null,
  webBaseUrl: string | null }
```

GET `/v1/github/install-url` 🔒 Auth

Get the OAuth URL to install the GitHub App for the authenticated user's account or organisation.

200 `{ installUrl: string }`

POST `/v1/github/setup/complete` 🔒 Auth

Link a GitHub App installation to the authenticated account after the user completes the GitHub App install flow.

PARAMETERS

Name	Type	Description
`installationId`	`string`	GitHub App installation ID

200

```
{ githubAppInstalled: boolean,
  installationId: string,
  redirectTo: string }
```

POST `/v1/github/repositories/validate` 🔒 Auth

Validate a GitHub repository URL for submission eligibility. Checks that the App has access and the user has push rights.

PARAMETERS

Name	Type	Description
`repoUrl`	`string`	Full GitHub repository URL

200

```
{ repoUrl, owner, name, fullName,
  defaultBranch, visibility: "public"|"private",
  permission: string }
```

POST **/v1/github/webhooks** 🔓 Public

Receive inbound GitHub webhook events (push, pull_request). Validates the HMAC-SHA256 signature using the webhook secret. Push events are linked to active submissions and trigger the verification worker.

200 `{ ok: true }  // processed`

204 No-op event (not a push to a tracked branch)

400 Invalid HMAC signature

FEATURE AREA

CLI Projects & Submissions

The project and submission endpoints are used exclusively by the `nibras` CLI. They handle project setup, task display, starter-kit download, and the full submission lifecycle.

Project Setup

GET `/v1/projects/:projectKey/manifest` 🔒 Auth

Fetch a project's manifest: test command, buildpack version, submission file paths, and allowed branch rules.

200

```
{ projectKey, releaseVersion, apiBaseUrl,
  buildpack?: { node?: { version } },
  test?:       { command, timeout? },
  submission?: { allowedPaths?, ignorePaths? } }
```

GET `/v1/projects/:projectKey/task` 🔒 Auth

Fetch project task instructions as Markdown. Displayed via `nibras task`.

200 `{ projectKey, task: string /* Markdown */ }`

GET `/v1/projects/:projectKey/starter-bundle` 🔒 Auth

Download the project starter kit as a ZIP archive (application/zip binary stream).

200 application/zip binary stream

POST `/v1/projects/:projectKey/setup` 🔒 Auth

Provision a GitHub repository for the student and apply the project template. Called once per student per project.

200

```
{ projectKey, manifest, task,
  repo: { owner, name, fullName, cloneUrl, defaultBranch },
  templateCloneUrl: string | null,
  starter: { type: "none" }
           | { type: "bundle", downloadUrl }
           | { type: "github-template", templateRepo } }
```

Submission Lifecycle

POST `/v1/submissions/prepare` 🔒 Auth

Create or reuse a submission attempt for a specific commit SHA. If a submission already exists for this commit, it is returned as-is.

PARAMETERS

Name	Type	Description
projectKey	string	Project identifier
commitSha	string	Full 40-char SHA
repoUrl	string	GitHub clone URL
branch	string	Branch name

200

```
{ submissionId: string, status: SubmissionStatus }
```

POST `/v1/submissions/:submissionId/local-test-result` 🔒 Auth

Record the outcome of a local test run before the student pushes. Stored for instructor visibility.

PARAMETERS

Name	Type	Description
exitCode	number	Process exit code (0 = pass)
summary	string	Test output summary
ranPrevious	boolean?	Whether re-running a previous test

200 `{ ok: true }`

GET `/v1/submissions/:submissionId` 🔒 Auth

Get the current status of a submission attempt. Polled by the CLI until a terminal status is reached.

200

```
{ submissionId, projectKey, commitSha, summary,
  status: "queued"|"running"|"passed"|"failed"|"needs_review",
  createdAt, updatedAt }
```

GET `/v1/submissions/:submissionId/stream` 🔒 Auth

Stream submission status updates as Server-Sent Events (SSE). Each event carries `{ status, summary }`. The stream closes on terminal status.

200 text/event-stream — events: `{ status, summary }`

GET `/v1/me/submissions` 🔒 Auth

List all submissions for the authenticated student (paginated). Supports `limit` and `offset` query parameters.

PARAMETERS

Name	Type	Description
`limit`	`number?`	Max records (1–200, default 50)
`offset`	`number?`	Records to skip (default 0)

200 StudentSubmission[] + X-Total-Count header

```
StudentSubmission {
  id, projectKey, milestoneId, commitSha, repoUrl, branch,
  status, summary, submissionType, submissionValue, notes,
  submittedAt, createdAt, updatedAt, localTestExitCode
}
```

FEATURE AREA

Tracking — Courses & Members

Course management endpoints. Instructors and TAs create and manage courses; students join via invite links.

Courses

GET `/v1/tracking/courses` 🔒 Auth

List all courses the authenticated user is enrolled in or teaches. Supports pagination.

200 CourseSummary[]

```
{ id, slug, title, termLabel, courseCode, isActive }
```

POST `/v1/tracking/courses` 🔒 Auth

Create a new course. Requires instructor or admin system role.

PARAMETERS

Name	Type	Description
slug	string	URL-safe identifier (e.g. "cs101-f26")
title	string	Display title
termLabel	string	e.g. "Fall 2026"
courseCode	string	e.g. "CS101"

201 CourseSummary

Members

GET `/v1/tracking/courses/:courseId/members` 🔒 Auth

List all members of a course with their role and level. Restricted to instructors and TAs.

200 CourseMember[]

```
{ id, courseId, userId, username, githubLogin,
  role: "student"|"instructor"|"ta", level, createdAt }
```

POST `/v1/tracking/courses/:courseId/members` 🔒 Auth

Add a member by GitHub login. Instructor or TA only.

PARAMETERS

Name	Type	Description
`githubLogin`	`string`	GitHub username
`role`	`string`	`"student" | "instructor" | "ta"`

201 CourseMember

DELETE **/v1/tracking/courses/:courseId/members/:userId** 🔒 Auth

Remove a member from the course. Instructor only.

200 `{ ok: true }`

PATCH **/v1/tracking/courses/:courseId/members/:userId/level** 🔒 Auth

Set a student's academic year level within the course (1–4).

PARAMETERS

Name	Type	Description
`level`	`1|2|3|4`	Academic year level

200 CourseMember

Invites

POST **/v1/tracking/courses/:courseId/invites** 🔒 Auth

Generate a time-limited invite link for the course.

PARAMETERS

Name	Type	Description
`role`	`string`	`"student" | "instructor" | "ta"`
`expiresInDays`	`number?`	Days until expiry (default 7)

200 `{ code: string, inviteUrl: string }`

POST **/v1/tracking/courses/:courseId/invites/bulk** 🔒 Auth

Generate multiple invite codes in a single call. Useful for batch student onboarding. Returns an array of invite records.

PARAMETERS

Name	Type	Description
count	number	Number of invites to create (1–50)
role	string?	"student" | "instructor" | "ta" (default "student")
maxUses	number?	Uses per code (default 1)
expiresAt	string?	ISO date string

201

```
{ invites: [{ code: string, inviteUrl: string, expiresAt: string | null }],
  count: number }
```

GET **/v1/tracking/invites/:code** 🔓 Public

Preview a course invite (course name, role) without joining. Used by the web UI to show a confirmation screen.

200

```
{ code, courseTitle, courseCode, termLabel,
  role, expiresAt: string | null }
```

POST **/v1/tracking/invites/:code/join** 🔒 Auth

Join a course using an invite code. Creates the membership record for the authenticated user.

200 `{ membership: CourseMember }`

Templates & Projects (Course-level)

GET **/v1/tracking/courses/:courseId/templates** 🔒 Auth

List reusable project templates available in this course.

200 ProjectTemplate[]

POST **/v1/tracking/courses/:courseId/templates** 🔒 Auth

Create a new project template in the course. Instructor or TA only.

201 ProjectTemplate

GET **/v1/tracking/courses/:courseId/projects** 🔒 Auth

List all published (and draft, for instructors) projects in the course.

PARAMETERS

Name	Type	Description
`limit`	`number?`	Max records
`offset`	`number?`	Skip N records

200 TrackingProjectSummary[]

GET **/v1/tracking/courses/:courseId/export.csv** 🔒 Auth

Export all student submission data for the course as CSV. Columns: githubLogin, username, milestoneTitle, projectKey, status, submittedAt, commitSha.

200 text/csv attachment

FEATURE AREA

Tracking — Projects & Milestones

Project and milestone management. Projects contain ordered milestones; students submit per milestone.

Projects

POST `/v1/tracking/projects` 🔒 Auth

Create a new project in a course. Starts as `draft` — not visible to students until published.

```
{ courseId, title, description?, projectKey,
  deliveryMode: "individual"|"team",
  level?, templateId?, teamSize?,
  gradeWeight?, startDate?, endDate? }
```

201 TrackingProjectSummary

GET `/v1/tracking/projects/:projectId` 🔒 Auth

Get full project details including all milestones, team formation status, and linked template.

200

```
{ id, projectKey, courseId, title, description,
  status: "draft"|"published"|"archived",
  deliveryMode, teamFormationStatus,
  applicationOpenAt, applicationCloseAt, teamLockAt,
  teamSize, gradeWeight, startDate, endDate,
  milestones: MilestoneSummary[],
  template: ProjectTemplateSummary | null }
```

PATCH `/v1/tracking/projects/:projectId` 🔒 Auth

Update a project's metadata. Accepts any subset of the create body.

200 TrackingProjectSummary

POST `/v1/tracking/projects/:projectId/publish` 🔒 Auth

Publish a draft project so students can view it and submit.

200 `{ ok: true, status: "published" }`

POST `/v1/tracking/projects/:projectId/unpublish` 🔒 Auth

Revert a published project to draft status. Hides it from students.

200 `{ ok: true, status: "draft" }`

Templates

GET `/v1/tracking/templates/:templateId` 🔒 Auth

Get full template details including defined roles, default milestones, and rubric criteria.

200 ProjectTemplate

PATCH `/v1/tracking/templates/:templateId` 🔒 Auth

Update a project template's roles, milestones, or rubric.

200 ProjectTemplate

Milestones

GET `/v1/tracking/projects/:projectId/milestones` 🔒 Auth

List all milestones for a project in display order.

200 MilestoneSummary[]

```
MilestoneSummary {
  id, projectId, title, description, order,
  dueAt, dueDateLabel, status, statusLabel, isFinal
}
```

POST `/v1/tracking/projects/:projectId/milestones` 🔒 Auth

Create a new milestone in the project. Instructor or TA only.

```
{ title, description?, order?,
  dueAt?, isFinal?: boolean }
```

201 MilestoneSummary

GET `/v1/tracking/milestones/:milestoneId` 🔒 Auth

Get milestone details together with all submission attempts and reviews for that milestone.

200

```
{ milestone: MilestoneSummary,
  submissions: SubmissionSummary[],
  reviews:     ReviewSummary[] }
```

PATCH **/v1/tracking/milestones/:milestoneId** 🔒 Auth

Update a milestone's title, due date, or other fields.

200 MilestoneSummary

DELETE **/v1/tracking/milestones/:milestoneId** 🔒 Auth

Delete a milestone. Only allowed when no submissions exist for it.

200 `{ ok: true }`

FEATURE AREA

Tracking — Submissions & Reviews

Endpoints for milestone submissions and instructor code reviews. Submissions can be GitHub-based, link-based, or text-based.

Submissions

GET `/v1/tracking/milestones/:milestoneId/submissions` 🔒 Auth

List all submissions for a milestone. Instructors and TAs see all; students see only their own.

PARAMETERS

Name	Type	Description
limit	number?	
offset	number?	

200 Submission[] + X-Total-Count

```
Submission {
  id, userId, projectId, projectKey, milestoneId,
  teamId, teamName, teamMemberUserIds[],
  commitSha, repoUrl, branch, status, summary,
  submissionType: "github"|"link"|"text",
  submissionValue, notes, submittedAt,
  createdAt, updatedAt, localTestExitCode
}
```

POST `/v1/tracking/milestones/:milestoneId/submissions` 🔒 Auth

Create a new submission for a milestone.

PARAMETERS

Name	Type	Description
submissionType	string	"github" | "link" | "text"
submissionValue	string	The submission content (URL, text, etc.)
notes	string?	Optional student notes
repoUrl	string?	Required for `github` type
branch	string?	Branch name
commitSha	string?	Commit SHA

201 Submission

GET `/v1/tracking/submissions/:submissionId` 🔒 Auth

Get a single submission by ID. Access is restricted to the submitter, team members, and instructors.

200 Submission

PATCH **/v1/tracking/submissions/:submissionId** 🔒 Auth

Update a submission (change repository URL, add notes, resubmit). Only allowed before the review is approved or graded.

200 Submission

DELETE **/v1/tracking/submissions/:submissionId** 🔒 Auth

Cancel a queued submission. Only allowed when the submission status is `queued`. The pending verification job is removed from the queue. Returns 409 if the submission is already running or has reached a terminal state.

200 `{ ok: true, status: "cancelled" }`

409 Submission is not in queued state

GET **/v1/tracking/submissions/:submissionId/commits** 🔒 Auth

Get GitHub push delivery events linked to this submission. Each record represents a webhook push event received from GitHub.

200 GithubDelivery[]

```
GithubDelivery {
  id, submissionId, repoUrl,
  eventType,           // "push", "pull_request" …
  deliveryId,          // GitHub delivery UUID
  ref,                 // "refs/heads/main"
  commitSha,
  payload: Record<string, unknown>,
  receivedAt
}
```

Reviews

GET **/v1/tracking/submissions/:submissionId/review** 🔒 Auth

Get the instructor review for a submission. Returns 404 if no review has been created yet.

200

```
Review {
  id, submissionId, reviewerUserId, status, score,
```

```
  status: "pending"|"approved"|"changes_requested"|"graded",
  feedback, rubric: [{ criterion, maxScore, earned? }],
  reviewedAt, createdAt, updatedAt,
  // AI grading (when enabled):
  aiConfidence?, aiNeedsReview?, aiReasoningSummary?,
  aiModel?, aiGradedAt?, aiCriterionScores?,
  aiEvidenceQuotes?
}
```

404 No review yet

POST `/v1/tracking/submissions/:submissionId/review` 🔒 Auth

Create or update the instructor review for a submission. Instructor or TA only. Idempotent — subsequent calls update the existing review.

PARAMETERS

Name	Type	Description
status	string	"pending" | "approved" | "changes_requested" | "graded"
score	number?	Numeric score
feedback	string?	Markdown feedback
rubric	array?	[{ criterion, maxScore, earned? }]

200 Review

GET `/v1/tracking/review-queue` 🔒 Auth

Get the instructor's pending review queue. Filterable by course, project, and status. Paginated.

PARAMETERS

Name	Type	Description
courseId	string?	Filter by course
projectId	string?	Filter by project
status	string?	Filter by submission status
limit	number?	
offset	number?	

200 `{ submissions: Submission[] }`

FEATURE AREA

Tracking — Team Formation

Role applications, automated team formation, and team management for group projects.

POST `/v1/tracking/projects/:projectId/applications` 🔒 Auth

Submit or update a role application for a team project. Students rank preferred roles; the formation algorithm uses these preferences.

PARAMETERS

Name	Type	Description
`statement`	`string?`	Optional motivation statement
`availabilityNote`	`string?`	Scheduling notes
`preferences`	`array`	[{ templateRoleId, rank }]

200 RoleApplication

GET `/v1/tracking/projects/:projectId/applications/me` 🔒 Auth

Get the current user's role application for a project.

200 RoleApplication

404 No application submitted yet

GET `/v1/tracking/projects/:projectId/applications` 🔒 Auth

List all role applications for a project. Instructor or TA only.

200 RoleApplication[]

POST `/v1/tracking/projects/:projectId/team-formation/generate` 🔒 Auth

Run the team formation algorithm over submitted applications. Returns a suggestion set — does not create teams yet. Safe to run multiple times.

PARAMETERS

Name	Type	Description
`algorithmVersion`	`string?`	Pin a specific algorithm version

200 TeamFormationRun

```
TeamFormationRun {
  id, projectId, algorithmVersion,
```

```
  result: {
    suggestions: [{ name, members: [{ userId, username,
                    level, roleKey, roleLabel }],
                    averageLevel }],
    waitlist:    [{ userId, username, level }]
  },
  createdByUserId, createdAt
}
```

POST `/v1/tracking/projects/:projectId/team-formation/lock` 🔒 Auth

Lock the suggested teams, creating permanent Team records. Once locked, team formation cannot be re-run.

PARAMETERS

Name	Type	Description
formationRunId	string?	Lock a specific run's suggestions

200 Team[]

GET `/v1/tracking/projects/:projectId/teams` 🔒 Auth

List all teams for a project. Visible to all course members.

200 Team[]

```
Team {
  id, projectId, name,
  status: "suggested"|"locked",
  lockedAt, members: TeamMember[],
  repo: TeamRepo | null,
  createdAt, updatedAt
}
```

PATCH `/v1/tracking/projects/:projectId/teams/:teamId` 🔒 Auth

Update a team's name or member role assignments. Instructor or TA only.

PARAMETERS

Name	Type	Description
name	string?	New team name
members	array?	[{ userId, roleKey, roleLabel }]

200 Team

FEATURE AREA

Tracking — Dashboards & Analytics

Aggregated views for students and instructors, plus per-student submission analytics.

GET `/v1/tracking/dashboard/home` 🔒 Auth

Role-aware home dashboard. Returns a student view, instructor view, or both, depending on the user's roles and the `mode` query param.

PARAMETERS

Name	Type	Description
mode	string?	"student" | "instructor"

200

```
{ availableModes: ("student"|"instructor")[],
  defaultMode,
  student?:    StudentDashboard,
  instructor?: InstructorDashboard }
```

GET `/v1/tracking/dashboard/student` 🔒 Auth

Detailed student project dashboard for a specific course — attention items, course snapshots, submission health, and upcoming milestones.

PARAMETERS

Name	Type	Description
courseId	string?	Filter to a specific course

200 StudentProjectsDashboardResponse

GET `/v1/tracking/dashboard/instructor` 🔒 Auth

Instructor overview dashboard — review queue summary, urgent items, per-course stats, and recent activity.

200 InstructorDashboardResponse

GET `/v1/tracking/dashboard/course/:courseId` 🔒 Auth

Full stats dashboard for a single course. Instructors, TAs, and admins only.

200 InstructorDashboardResponse

GET /v1/tracking/analytics/instructor

🔒 Auth

Aggregated course analytics for an instructor — total students, submission counts, pass rate, per-milestone breakdown, and per-student progress. Requires course management permission.

PARAMETERS

Name	Type	Description
courseId	string	Required — course to analyse

200

```
{ courseId, courseTitle,
  totalStudents: number,
  submissionCount: number,
  passRate: number,           // 0.0 – 1.0
  milestones: [{
    milestoneId, milestoneTitle,
    submissionCount, passCount, failCount, needsReviewCount
  }],
  students: [{
    userId, username,
    submittedMilestones, passedMilestones, totalMilestones
  }]
}
```

GET /v1/tracking/analytics/student

🔒 Auth

Per-course, per-project, per-milestone submission analytics for the authenticated student. Used to power the Submissions page progress bars.

PARAMETERS

Name	Type	Description
courseId	string?	Filter to a specific course

200

```
{ userId,
  analytics: [{
    courseId, courseTitle,
    projects: [{
      projectId, projectTitle,
      totalMilestones, submittedMilestones, passedMilestones,
      milestones: [{
        milestoneId, milestoneTitle, dueAt,
        submissionCount,
        latestStatus: "queued"|"running"|"passed"|"failed"|"needs_review"|null,
        latestSubmittedAt
      }]
    }]
  }]
}
```

GET `/v1/tracking/activity` 🔒 Auth

Activity feed for the current user — recent events across all their courses (submissions, reviews, membership changes).

200 ActivityEvent[]

```
ActivityEvent {
  id, actorUserId, courseId, projectId,
  milestoneId, submissionId,
  action, summary, createdAt
}
```

FEATURE AREA

Program Planning

Academic program management: programs, versions, catalog courses, requirement groups, specialization tracks, student plans, petitions, and approvals.

Programs

GET `/v1/programs` 🔒 Auth

List all published academic programs.

200 ProgramSummary[]

```
ProgramSummary {
  id, slug, title, code, academicYear,
  totalUnitRequirement,
  status: "draft"|"published"|"archived",
  activeVersionId, createdAt, updatedAt
}
```

POST `/v1/programs` 🔒 Auth

Create a new academic program. Program manager role required.

PARAMETERS

Name	Type	Description
slug	string	URL-safe identifier
title	string	Display title
code	string	e.g. "BSc-CS"
academicYear	string	e.g. "2026–27"
totalUnitReq	number?	Required credit units
status	string?	"draft" | "published" | "archived"

201 ProgramSummary

GET `/v1/programs/:programId/versions/:versionId` 🔒 Auth

Get the full detail of a program version including tracks, catalog courses, and requirement groups.

200

```
{ program: ProgramSummary,
  version: VersionSummary,
  tracks: TrackSummary[],
```

```
  catalogCourses: CatalogCourse[],
  requirementGroups: RequirementGroup[] }
```

POST `/v1/programs/:programId/versions` 🔒 Auth

Create a new version of a program (e.g. for a new academic year).

201 VersionSummary

POST `/v1/programs/:programId/catalog-courses` 🔒 Auth

Add a course to the program's catalog of eligible courses.

PARAMETERS

Name	Type	Description
subjectCode	string	e.g. "CS"
catalogNumber	string	e.g. "161"
title	string	Course title
defaultUnits	number	Credit units
department	string	Offering department

201 CatalogCourse

Requirement Groups & Tracks

POST `/v1/programs/:programId/requirement-groups` 🔒 Auth

Create a requirement group (foundation, core, depth, elective, capstone, or policy) in a program version.

```
{ title, category: "foundation"|"core"|"depth"|"elective"|"capstone"|"policy",
  trackId?, minUnits?, minCourses?, notes?, sortOrder?, noDoubleCount? }
```

201 RequirementGroup

PATCH `/v1/programs/:programId/requirement-groups/:groupId` 🔒 Auth

Update a requirement group's fields or rules.

200 RequirementGroup

POST `/v1/programs/:programId/tracks` 🔒 Auth

Create a specialization track in a program version.

PARAMETERS

Name	Type	Description
slug	string	URL-safe identifier
title	string	Display title
desc	string?	Description
selectionYearStart	number?	Earliest year for selection

201 TrackSummary

PATCH /v1/programs/:programId/tracks/:trackId 🔒 Auth

Update a specialization track's title, description, or selection year.

200 TrackSummary

Student Plan

POST /v1/programs/:programId/enroll 🔒 Auth

Enroll the authenticated student in an academic program. Creates a StudentProgramPlan.

200 StudentProgramPlan

GET /v1/programs/student/me 🔒 Auth

Get the current student's full program plan: selected track, planned courses, requirement decisions, petitions, and approvals.

200 StudentProgramPlan

```
StudentProgramPlan {
  id, userId, program, version, selectedTrack,
  availableTracks, status, isLocked, canSelectTrack,
  catalogCourses[], requirementGroups[],
  plannedCourses[], decisions[], petitions[], approvals[],
  latestSheet: { generatedAt } | null
}
```

PATCH /v1/programs/student/me/plan 🔒 Auth

Update the student's planned courses (called on every drag-and-drop interaction in the planner UI).

```
{ plannedCourses: [{
    catalogCourseId, plannedYear, plannedTerm: "fall"|"spring",
    sourceType?: "standard"|"transfer"|"petition"|"manual",
```

```
    note?
  }] }
```

200 StudentProgramPlan

GET **/v1/programs/student/me/recommend-track** 🔒 Auth

Get track recommendations based on the student's Year 1 planned courses. Score is the fraction of unit overlap between planned courses and each track's requirements.

200

```
{ recommendations: [{
    trackId, trackTitle, trackSlug, trackDescription,
    matchScore: number,     // 0.0 – 1.0
    matchedUnits, totalTrackUnits, matchedCourseCount,
    reason: string
  }],
  year1CourseCount: number }
```

POST **/v1/programs/student/me/select-track** 🔒 Auth

Formally select a specialization track for the student's program. Can only be done when `canSelectTrack` is true on the plan.

PARAMETERS

Name	Type	Description
`trackId`	`string`	Track CUID to select

200 StudentProgramPlan

Program Sheet

GET **/v1/programs/student/me/sheet** 🔒 Auth

Get the printable program sheet view — all requirement groups with matched and unmatched courses, petition summaries, and approval status.

200 ProgramSheetView

POST **/v1/programs/student/me/generate-sheet** 🔒 Auth

Snapshot and regenerate the printable program sheet. Useful before advisor review.

200 ProgramSheetView

Petitions

POST `/v1/programs/student/me/petitions` 🔒 Auth

Submit a petition (transfer credit, course substitution, or requirement waiver).

PARAMETERS

Name	Type	Description
`type`	`string`	`"transfer_credit" | "substitution" | "waiver"`
`justification`	`string`	Reason for petition
`targetGroupId`	`string?`	Requirement group this affects
`originalId`	`string?`	Original catalog course ID
`substituteId`	`string?`	Substitute catalog course ID

201 Petition

GET `/v1/programs/student/me/petitions` 🔒 Auth

List the current student's petitions.

200 Petition[]

GET `/v1/programs/:programId/petitions` 🔒 Auth

List all petitions for a program. Program managers only.

200 Petition[]

PATCH `/v1/programs/:programId/petitions/:petitionId` 🔒 Auth

Approve or reject a petition. Program manager only.

PARAMETERS

Name	Type	Description
`status`	`string`	`"approved" | "rejected"`
`reviewerNotes`	`string?`	Optional reviewer notes

200 Petition

Approvals

POST `/v1/programs/:programId/approvals/:studentProgramId/advisor` 🔒 Auth

Record advisor-stage approval or rejection for a student's program plan.

PARAMETERS

Name	Type	Description
status	string	"approved" | "rejected"
notes	string?	Optional notes

200 ProgramApproval

POST /v1/programs/:programId/approvals/:studentProgramId/department 🔒 Auth

Record department-stage approval or rejection for a student's program plan.

PARAMETERS

Name	Type	Description
status	string	"approved" | "rejected"
notes	string?	Optional notes

200 ProgramApproval

FEATURE AREA

Notifications

In-app notification endpoints. The web dashboard polls `/count` every 60 seconds to show the unread badge.

GET `/v1/notifications` 🔒 Auth

List the latest 50 notifications for the authenticated user, newest first.

200

```
{ notifications: [{
    id, type, title, body,
    link: string | null,
    read: boolean,
    createdAt
  }] }
```

GET `/v1/notifications/count` 🔒 Auth

Return the number of unread notifications. Polled every 60 s by the web dashboard header.

200 `{ count: number }`

POST `/v1/notifications/read-all` 🔒 Auth

Mark all of the authenticated user's notifications as read.

200 `{ ok: true }`

PATCH `/v1/notifications/:id/read` 🔒 Auth

Mark a single notification as read by ID. Returns 404 if the notification does not belong to the authenticated user.

200 `{ ok: true }`

404 Notification not found or not owned by user

Notification Preferences

GET `/v1/notifications/preferences` 🔒 Auth

List all notification preference settings for the authenticated user. Returns one record per notification type that has been explicitly configured; unset types default to enabled.

200

```
{ preferences: [{
    id, userId,
    type: string,        // e.g. "feedback", "review_ready"
    enabled: boolean,
    createdAt, updatedAt
  }] }
```

PATCH **/v1/notifications/preferences/:type** 🔒 Auth

Enable or disable a specific notification type for the authenticated user. Upserts — creates the preference record if it does not yet exist.

PARAMETERS

Name	Type	Description
enabled	boolean	true to enable, false to silence

200

```
{ id, userId, type, enabled, createdAt, updatedAt }
```

FEATURE AREA

Admin

Platform-wide admin endpoints. All require `systemRole: "admin"`.

Submission Admin

GET `/v1/admin/submissions` 🔒 Auth

List all submissions platform-wide. Filter by status or project.

PARAMETERS

Name	Type	Description
status	string?	Submission status filter
projectId	string?	Filter by project

200 Submission[]

PATCH `/v1/admin/submissions/:submissionId/status` 🔒 Auth

Override a submission's verification status. Use to correct false-positive/negative automated results.

PARAMETERS

Name	Type	Description
status	string	"passed" | "failed" | "needs_review"
summary	string?	Optional override message

200 Submission

GET `/v1/admin/submissions/:submissionId/logs` 🔒 Auth

Retrieve the raw verification worker logs for a submission.

200 `{ logs: string }`

POST `/v1/admin/submissions/:submissionId/retry` 🔒 Auth

Re-queue a submission for fresh automated verification. Resets status to `queued`.

200 `{ ok: true, status: "queued" }`

POST `/v1/admin/submissions/bulk-retry` 🔒 Auth

Re-queue multiple submissions for fresh verification in a single request. Each submission is reset to `queued` and a new verification job is enqueued. Silently skips IDs that are already queued or running.

PARAMETERS

Name	Type	Description
submissionIds	string[]	Array of submission CUIDs to retry

200

```
{ ok: true, queued: number, skipped: number }
```

Audit Log

GET /v1/admin/audit-logs 🔒 Auth

Read the platform audit log. Every significant write action (course creation, role changes, submission overrides, etc.) is recorded automatically. Supports filtering and offset pagination.

PARAMETERS

Name	Type	Description
action	string?	Filter by action name (substring match)
targetType	string?	Entity type (e.g. "Course", "Submission")
courseId	string?	Filter by course
userId	string?	Filter by acting user
fromDate	string?	ISO date lower bound
toDate	string?	ISO date upper bound
limit	number?	Max records (default 50, max 200)
offset	number?	Records to skip

200

```
{ logs: [{
    id, action, targetType, targetId,
    userId: string | null,
    courseId: string | null,
    metadata: Record<string, unknown>,
    createdAt
  }],
  total: number }
```

Platform Management

GET /v1/admin/projects 🔒 Auth

List all projects across all courses on the platform.

200 TrackingProjectSummary[]

DELETE **/v1/admin/courses/:courseId** 🔒 Auth

Permanently delete a course and all its data. Irreversible.

200 `{ ok: true }`

Warning: This permanently deletes the course, all its projects, milestones, and submission records. There is no recovery path.

POST **/v1/admin/projects/:projectId/archive** 🔒 Auth

Archive a project. Students can no longer submit; historical data is preserved.

200 `{ ok: true, status: "archived" }`

User Management

GET **/v1/admin/users** 🔒 Auth

List all platform users.

200 UserRecord[]

```
UserRecord {
  id, username, email, githubLogin,
  githubLinked, githubAppInstalled,
  systemRole: "user"|"admin", yearLevel
}
```

PATCH **/v1/admin/users/:userId/role** 🔒 Auth

Promote or demote a user's system role.

PARAMETERS

Name	Type	Description
role	string	"user" | "admin"

200 UserRecord

GET **/v1/admin/students** 🔒 Auth

List all students with their global academic year level.

200 StudentRecord[]

```
StudentRecord { userId, username, githubLogin, yearLevel }
```

PATCH **/v1/admin/students/:userId/year** 🔒 Auth

Set a student's global academic year level (1–4). This is separate from the per-course level.

PARAMETERS

Name	Type	Description
`yearLevel`	`1|2|3|4`	Global year level

200 StudentRecord

Nibras Platform · REST API Reference v2.0.0 · May 2026
97 endpoints · 12 feature areas
Generated from `packages/contracts/src/api-reference.ts`

Internal documentation — not for public distribution